



8. Python: Functions

Write Once, Call Anywhere.

Previous Section:

 [7. Python: Loops](#)

Contents:

[1. What is a Function?](#)

[2. Properties of Functions](#)

[Printing Functions](#)

[Returning Functions](#)

[3. Define a Function](#)

[Example: Odd and Even Checker](#)

[Example: Prime Number Checker](#)

[Example: Recursive Function](#)

[Why Functions are Good](#)

[4. Additional Concepts in Defining Functions](#)

[Overriding Built-in Functions](#)

[Multiple Functions with the Same Name](#)

[Nested Functions](#)

[Using `print\(\)` and `return` Together](#)

[Variables with Functions](#)

[Using Functions Inside Loops](#)

[Returning Multiple Values](#)

[Returning Values as a Collection](#)

[Summary](#)

[References](#)

Functions are one of the most important concepts in programming. They allow a programmer to write a block of code once and reuse it whenever needed, making programs more organized, efficient, and easier to maintain.

To Iris, functions were more than just reusable code. She viewed them as specialized routines within a system, where each function had a single responsibility and performed its assigned task only when called upon. Instead of rewriting the same logic repeatedly, a function could preserve a procedure and execute it whenever the situation demanded it.

Functions also improve readability. A large program can be divided into smaller meaningful sections, making it easier to understand and debug. Python itself already contains many built-in functions such as `len()`, `sum()`, `min()`, and `max()`, each created to solve a specific purpose.

However, programmers can also create their own custom functions, known as user-defined functions. In many cases, a function requires additional information in order to work properly. These are called parameters. Parameters act as placeholders for the values a function needs, whether it is a number, string, list, or other data type. Some functions may not require parameters at all, while others depend on them entirely.

1. What is a Function?

A function is a block of code designed to perform a specific task. The code inside the function will not execute immediately; instead, it only runs when the function is called.

Iris often described functions as isolated procedures inside a larger system. Each function had a purpose, an input, and an expected output. Some existed only to display information, while others calculated and returned values for future use. By separating logic into functions, a programmer can reduce repetition and make programs significantly cleaner.

Functions in Python generally follow this structure:

```
def function_name(parameters):  
    # code
```

The keyword `def` is used to define a function. The function name identifies the routine, while parameters specify what information is needed for execution.

For example:

```
def greet():  
    print("Hello World from Python")  
  
greet()
```

Hello World from Python

This function has no parameters because it always performs the same action. However, many functions require input:

```
def greet(name):  
    print(f"Hello World, {name}")  
  
greet("Alice")
```

Hello World Alice

Here, `name` becomes the parameter. Whenever the function is called, the value provided will replace `name`.

To Iris, this behavior resembled how systems adapt to different scenarios using the same core procedure. The structure remains identical, but the input changes the result.

2. Properties of Functions

Functions are versatile and can behave differently depending on how they are designed. In Python, user-defined functions are commonly separated into two major categories:

- Functions that simply print results.

- Functions that return values.

Although both perform tasks, they are used differently.

Printing Functions

A printing function directly displays information to the screen. Its purpose is immediate output rather than storing a result for later use.

```
def calculate(x, y):  
    print(x + y)  
    print(x - y)  
    print(x * y)  
    print(x / y)  
  
calculate(x=12, y=25)  
calculate(12.43, 9.11)
```

```
37  
-13  
300  
0.48
```

Functions can receive arguments in multiple ways: Positional arguments or Keyword arguments. However, Python requires positional arguments to come first.

```
calculate(12, y=25)    # Valid  
calculate(x=12, 25)    # Invalid
```

I considered this rule important because Python must always know which value belongs to which parameter without ambiguity.

Another important detail is that printing functions do not actually give back a value. Because of this, if you attempt:

```
print(calculate(12, 25))
```

the output after the calculations will become: `None` . This happens because the function only prints and does not return anything.

Returning Functions

A returning function sends a value back to the caller using the `return` keyword. This allows the result to be stored, reused, or processed further.

```
def display_info(name, age, city):  
    return f"Hello {name}, {age}, from {city}"  
  
name, age, city = 'Alice', 23, 'Washington D.C.'  
  
print(display_info(name, age, city))
```

Hello Alice, 23, from Washington D.C.

Unlike printing functions, returning functions are far more flexible because their outputs can be reused inside variables, calculations, loops, or conditions.

```
def area_rectangle(a, b):  
    return a * b  
  
print(f"The area of rectangle is: {area_rectangle(12, 2  
5)}")
```

The area of rectangle is: 300

I viewed returning functions as more modular and reusable because they separate computation from display. The function focuses only on producing a result, while the caller decides how that result should be used.

Another important property is that `return` immediately terminates the function. Once Python reaches a `return` statement, the function stops executing further

lines. In some cases, this behavior can effectively act similarly to breaking out of a process early.

3. Define a Function

Defining a function means creating a reusable procedure that performs a task whenever called. A well-defined function usually focuses on one specific responsibility.

Example: Odd and Even Checker

```
def is_odd_even(x):  
    if x % 2 == 0:  
        print(x, "is an even number")  
    else:  
        print(x, "is an odd number")  
  
number = int(input("Enter a number: "))  
is_odd_even(number)
```

Enter a number: 54
54 is an even number

This function determines whether a number is odd or even using the modulus operator `%`. If the remainder after division by 2 is zero, the number is even; otherwise, it is odd.

This is one of the simplest examples of decision-making encapsulated into a reusable routine.

Example: Prime Number Checker

Functions become more useful when solving larger problems.

```

def is_prime(x):
    is_prime = True

    if x < 0:
        is_prime = False
    elif x == 0 or x == 1:
        is_prime = False
    elif x == 2:
        is_prime = True
    else:
        for i in range(2, x):
            if x % i == 0:
                is_prime = False
                break

    return is_prime

number = int(input("Enter a number: "))

if is_prime(number):
    print(number, "is a prime number")
else:
    print(number, "is not a prime number")

```

Enter a number: 29
 29 is a prime number

This function demonstrates several important concepts: Conditional statements; Loops; Boolean values; Returning results

The function checks every number from 2 up to $x - 1$. If any value divides x perfectly, then the number is not prime. The `break` statement immediately stops the loop once a divisor is found, improving efficiency. I described this process as narrowing down possibilities until certainty was achieved.

Example: Recursive Function

Functions can even call themselves. This technique is known as recursion.

The Fibonacci sequence follows the pattern: $F(0) = 0$; $F(1) = 1$; $F(n) = F(n-1) + F(n-2)$

This relationship can be expressed naturally using recursion:

```
def fibonacci(x):  
    if x == 0:  
        return 0  
    elif x == 1:  
        return 1  
    else:  
        return fibonacci(x - 1) + fibonacci(x - 2)  
  
print(fibonacci(10))
```

55

The function repeatedly calls itself until it eventually reaches the base cases (0 or 1).

Recursion represented a system solving a problem by continuously reducing it into smaller versions of itself until the answer became simple enough to resolve directly.

Why Functions are Good

Functions make programs reusable, modular, and easier to maintain. Instead of rewriting the same logic repeatedly, programmers can isolate procedures into dedicated routines and execute them whenever needed.

They also improve readability by dividing large systems into smaller understandable components. Complex tasks become easier to debug, modify, and expand because each function focuses on a single responsibility.

Functions were not merely programming syntax. They were a way to structure logic itself — transforming scattered instructions into organized systems capable of scaling far beyond a single script.

4. Additional Concepts in Defining Functions

Functions in Python are flexible, but this flexibility also introduces several important behaviors that programmers must understand. Iris considered these behaviors essential because they reveal how Python prioritizes, replaces, and manages procedures internally.

Overriding Built-in Functions

Python already contains many built-in functions such as `len()`, `sum()`, and `max()`. However, it is technically possible to define your own function using the same name as a built-in function.

When this happens, the original built-in function becomes overridden inside the current scope. Python will now use your new definition instead of the original one.

```
def len(numbers):  
    total = 0  
    for i in numbers:  
        total += i  
  
    return total  
  
values = [1, 2, 3, 4, 5]  
print(len(values))
```

15

Normally, the built-in `len()` counts the number of elements in a list. But after redefining it, the function now calculates the sum instead.

This concept resembles overriding in Object-Oriented Programming (OOP), where a newer implementation replaces an older behavior. Overriding demonstrated that definitions in programming are not permanent. The latest valid instruction becomes the active truth of the system.

Multiple Functions with the Same Name

If two functions share the same name, Python will only keep the most recent definition. The previous version is completely replaced.

```
def add(a, b):  
    return a + b  
  
# Python will execute this one  
def add(a, b):  
    return str(a) + str(b)  
  
print(add(10, 20))
```

1020

The second function overwrites the first one. Python ignores the earlier definition because the newest version becomes the active implementation.

Unlike some other programming languages, Python does not support traditional function overloading purely by redefining the same name with different parameter types. Python valuing clarity over ambiguity. A function name can only point to one procedure at a time.

Nested Functions

Functions can also exist inside other functions. These are called nested functions.

Nested functions are different from recursive functions because the inner function does not necessarily call itself. Instead, the inner function exists only within the outer function.

```
def create_student():

    def get_name():
        return "Alice"

    def get_background():
        return "Computer Science"

    student_name = get_name()
    background = get_background()

    return f"{student_name} studies {background}"

print(create_student())
```

Alice studies Computer Science

Nested functions are useful for:

- Breaking large problems into smaller procedures
- Organizing complex logic
- Solving numerical and computational problems
- Restricting helper functions so they are only accessible internally

Nested functions resembled subsystems inside a larger system. Each smaller procedure existed only to support the primary operation.

Using `print()` and `return` Together

A function can both print information and return a value at the same time.

```
def calculate_square(x):  
    print("Calculating square...")  
    return x ** 2  
  
result = calculate_square(5)  
print(result)
```

```
Calculating square...  
25
```

The `print()` statement always executes whenever the function is called, while the `return` statement sends a value back to the caller.

This is important because many beginners assume a function must either print or return, but it can perform both actions simultaneously. Regardless, I would view printing as immediate communication, while returning represented transferring data for future use.

Variables with Functions

The returned value of a function can be stored inside a variable.

```
def compute_salary():  
    return 5000  
  
salary = compute_salary()  
  
print(salary)  
print(salary ** 2)
```

```
5000  
25000000
```

Because `return` gives back a value, that value can later participate in calculations, conditions, loops, or assignments. This is why returning functions are considered more reusable and versatile compared to functions that only print.

Using Functions Inside Loops

Functions are often combined with loops to solve repetitive problems.

For example, a prime-checking function can be reused to generate a list of prime numbers from 1 to 50.

```
def is_prime(x):  
    if x < 2:  
        return False  
  
    for i in range(2, x):  
        if x % i == 0:  
            return False  
  
    return True  
  
prime_numbers = []  
  
for i in range(1, 51):  
    if is_prime(i):  
        prime_numbers.append(i)  
  
print(prime_numbers)
```

[2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]

Here, the function acts as a reusable validator, while the loop repeatedly applies it to multiple values. This combination represented automation at scale: a single procedure repeatedly executed across an entire dataset without rewriting the logic each time.

Returning Multiple Values

Functions in Python are not limited to returning only a single value. A function can return multiple values at the same time, allowing a programmer to solve more complex problems in a clean and organized way. This behavior resembles a system producing several pieces of processed information simultaneously instead of forcing everything into a single output.

Imagine we want to calculate the square of three numbers separately, then combine them later in the main program.

```
def compute_squares(a, b, c):  
    square_a = a ** 2  
    square_b = b ** 2  
    square_c = c ** 2  
  
    return square_a, square_b, square_c  
  
x, y, z = compute_squares(2, 3, 4)  
  
total = x + y + z  
print("The total is:", total)
```

29

Here, the function returns three separate values at once. Python automatically packs them together and allows them to be unpacked into variables. Once the values are returned, they can be reused in calculations.

Returning Values as a Collection

Python is internally returning the values as a tuple.

```
def compute_squares(a, b, c):  
    return a ** 2, b ** 2, c ** 2  
  
values = compute_squares(2, 3, 4)  
print(values)
```

(4, 9, 16)

This means the returned data can also be stored as a single collection instead of separate variables. Returning multiple values represented efficiency in system design. Instead of repeatedly calling separate functions for related computations, a single procedure could process and distribute multiple results at once.

Summary

Functions are reproducible, reusable, and highly versatile. They allow programmers to organize logic into smaller manageable procedures that can be executed repeatedly without rewriting code.

Beyond simple reuse, functions can override existing behavior, exist inside other functions, interact with loops and conditions, return reusable data, and even cooperate with larger systems of logic.

Functions are not simply reusable code blocks. They are modular units of behavior — isolated systems capable of processing information, communicating results, and scaling complex programs into structured and maintainable architectures.

References

<https://www.geeksforgeeks.org/python/python-functions/>

<https://www.datacamp.com/tutorial/functions-python-tutorial>

<https://www.codecademy.com/learn/flask-introduction-to-python/modules/learn-python3-functions/cheatsheet>

Next Section:

</> 9. Python: Local and Global Variables